

苏州大学实验报告

院系	计算机学院	年级专业	21 计科	姓名	方浩楠	学号	2127405048
课程名称	操作系统课程实践					成绩	
指导教师	王红玲	同组实验者	无	实验日期			

实验名称 实验 5 系统调用

一. 实验目的

在理解系统调用基本原理的基础上, 通过修改 Linux 内核相关内容, 新增一个系统调用, 由此掌握 Linux 系统下系统调用的实现和调用过程。

二. 实验内容

为 Linux 新增一个系统调用, 重新编译内核并安装, 并编写 C 语言程序调用该系统调用。可选用教材上提供的两种方法之一完成: 内核编译法或内核模块法。

具体要求: ①完成一个简单系统调用 (无参数)。②完成一个带参系统调用函数。

三. 实验步骤和结果

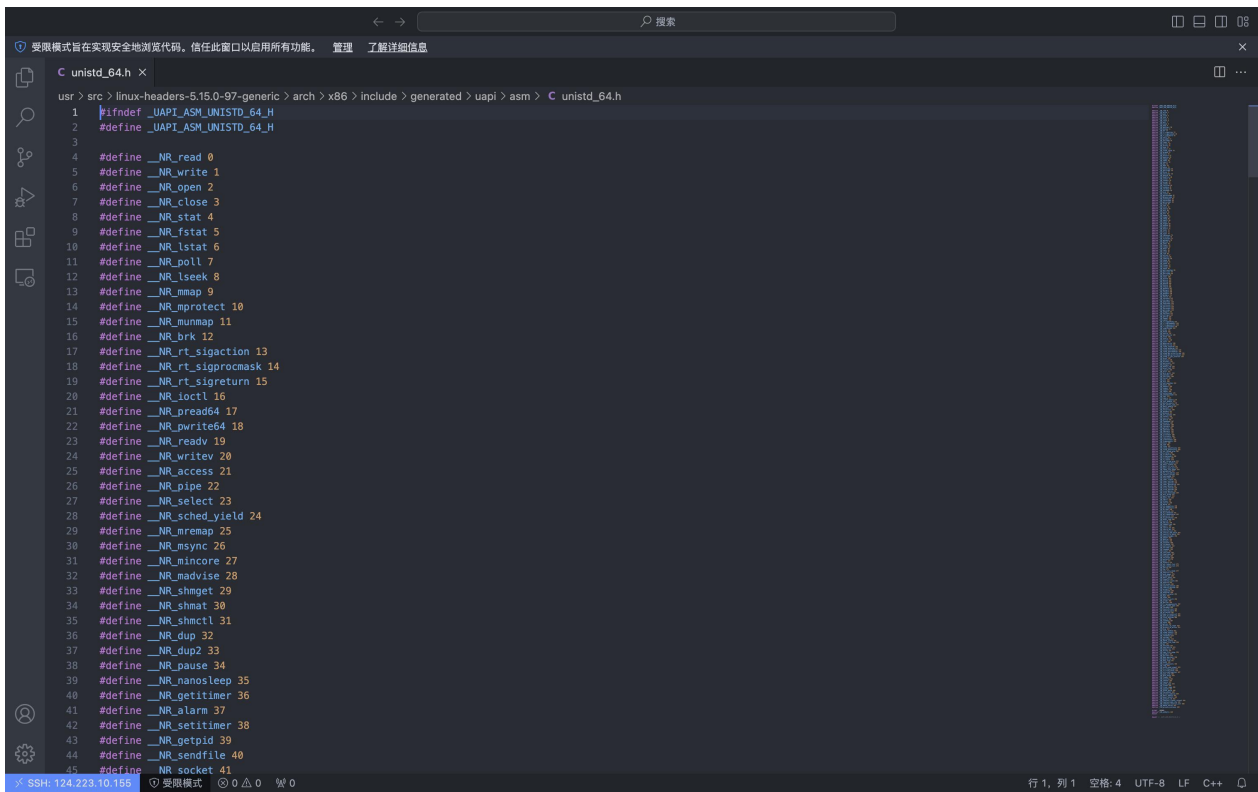
首先查看系统当前内核版本,使用 `uname -r`

然后查找系统调用号

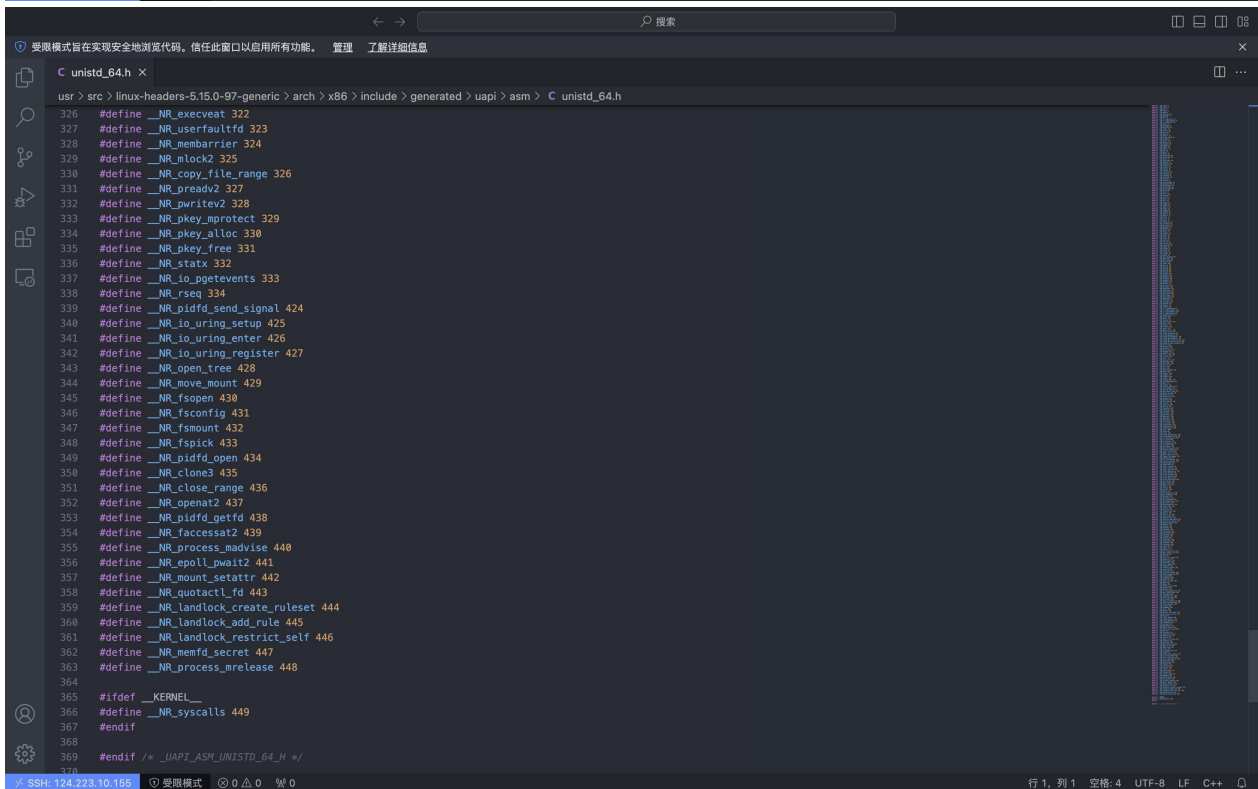
输入 `sudo find / -name unistd_64.h`

```
ubuntu@ubuntu:~$ sudo find / -name unistd_64.h
/usr/src/linux-headers-5.15.0-101-generic/arch/x86/include/generated/uapi/asm/unistd_64.h
/usr/src/linux-headers-5.15.0-97-generic/arch/x86/include/generated/uapi/asm/unistd_64.h
/usr/include/x86_64-linux-gnu/asm/unistd_64.h
```

发现位置为usr/src/linux-headers-5.15.0-97-generic/arch/x86/include/generated/uapi/asm/unistd_64.h
打开该文件



```
C unistd_64.h X
usr > src > linux-headers-5.15.0-97-generic > arch > x86 > include > generated > uapi > asm > C unistd_64.h
1  #ifndef _UAPI_ASM_UNISTD_64_H
2  #define _UAPI_ASM_UNISTD_64_H
3
4  #define _NR_read 0
5  #define _NR_write 1
6  #define _NR_open 2
7  #define _NR_close 3
8  #define _NR_stat 4
9  #define _NR_fstat 5
10 #define _NR_lstat 6
11 #define _NR_poll 7
12 #define _NR_lseek 8
13 #define _NR_mmap 9
14 #define _NR_mprotect 10
15 #define _NR_munmap 11
16 #define _NR_brk 12
17 #define _NR_rt_sigaction 13
18 #define _NR_rt_sigprocmask 14
19 #define _NR_rt_sigreturn 15
20 #define _NR_ioctl 16
21 #define _NR_pread64 17
22 #define _NR_pwrite64 18
23 #define _NR_readv 19
24 #define _NR_writev 20
25 #define _NR_access 21
26 #define _NR_pipe 22
27 #define _NR_select 23
28 #define _NR_sched_yield 24
29 #define _NR_mremap 25
30 #define _NR_msync 26
31 #define _NR_mincore 27
32 #define _NR_madvise 28
33 #define _NR_shmget 29
34 #define _NR_shmat 30
35 #define _NR_shmctl 31
36 #define _NR_dup 32
37 #define _NR_dup2 33
38 #define _NR_pause 34
39 #define _NR_nanosleep 35
40 #define _NR_getitimer 36
41 #define _NR_alarm 37
42 #define _NR_setitimer 38
43 #define _NR_getpid 39
44 #define _NR_sendfile 40
45 #define _NR_socket 41
```



```
C unistd_64.h X
usr > src > linux-headers-5.15.0-97-generic > arch > x86 > include > generated > uapi > asm > C unistd_64.h
326 #define _NR_execveat 322
327 #define _NR_userfaultfd 323
328 #define _NR_membarrier 324
329 #define _NR_mlock2 325
330 #define _NR_copy_file_range 326
331 #define _NR_preadv2 327
332 #define _NR_pwritev2 328
333 #define _NR_key_mprotect 329
334 #define _NR_key_alloc 330
335 #define _NR_key_free 331
336 #define _NR_statx 332
337 #define _NR_io_pgetevents 333
338 #define _NR_rseq 334
339 #define _NR_pidfd_send_signal 424
340 #define _NR_io_uring_setup 425
341 #define _NR_io_uring_enter 426
342 #define _NR_io_uring_register 427
343 #define _NR_open_tree 428
344 #define _NR_move_mount 429
345 #define _NR_fsopen 430
346 #define _NR_fsconfig 431
347 #define _NR_fsmount 432
348 #define _NRfspick 433
349 #define _NR_pidfd_open 434
350 #define _NR_clone3 435
351 #define _NR_close_range 436
352 #define _NR_openat2 437
353 #define _NR_pidfd_getfd 438
354 #define _NR_faccessat2 439
355 #define _NR_process_madvise 440
356 #define _NR_epoll_pwait2 441
357 #define _NR_mount_setattr 442
358 #define _NR_quotactl_fd 443
359 #define _NR_landlock_create_ruleset 444
360 #define _NR_landlock_add_rule 445
361 #define _NR_landlock_restrict_self 446
362 #define _NR_memfd_secret 447
363 #define _NR_process_mrelease 448
364
365 #ifdef _KERNEL_
366 #define _NR_syscalls 449
367 #endif
368
369 #endif /* _UAPI_ASM_UNISTD_64_H */
```

发现系统调用号截止到 449,因此选择了 500 和 501

查找 sys_call_table 地址

输入 `sudo cat /proc/kallsyms | grep sys_call_table`

```
ubuntu@VM-4-12-ubuntu ~/os_experiment/week5$ sudo cat /proc/kallsyms | grep sys_call_table
ffffffffa0800320 D sys_call_table
ffffffffa0801440 D ia32_sys_call_table
ffffffffa0802260 D x32_sys_call_table
ffffffffffc0937380 b sys_call_table [syscall]
```

发现 sys_call_table 地址为 ffffffff0800320

然后编写代码

```

1  #include <linux/module.h>
2  #include <linux/kernel.h>
3  #include <linux/init.h>
4  #include <linux/unistd.h>
5  #include <linux/time.h>
6  #include <asm/uaccess.h>
7  #include <linux/sched.h>
8  #include <linux/kallsyms.h>
9  #include <linux/types.h>
10
11 #define __NR_syscall 500
12 #define __NR_syscall_with_param 501
13 #define SYS_CALL_TABLE_ADDRESS 0xfffffffffa0800320ULL
14
15 unsigned long *sys_call_table;
16
17 unsigned int clear_and_return_cr0(void);
18 void setback_cr0(unsigned int val);
19
20 static int sys_mycall(void);
21 static long sys_mycall_with_param(int param);
22
23 int orig_cr0; /* 用来存储cr0寄存器原来的值 */
24 static int (*anything_saved)(void); /* 用来保存原始系统调用 */
25 static long (*anything_saved_with_param)(int); /* 用来保存原始的带参系统调用 */
26
27 unsigned int clear_and_return_cr0(void) {
28     unsigned int cr0 = 0;
29     unsigned int ret;
30     asm volatile ("movq %%cr0, %%rax" : "=a"(cr0));
31     ret = cr0;
32     cr0 &= 0xffffefff; /* 将cr0变量值中的第17位清0 */
33     asm volatile ("movq %%rax, %%cr0" :: "a"(cr0));
34     return ret;
35 }
36
37 void setback_cr0(unsigned int val) {
38     asm volatile ("movq %%rax, %%cr0" :: "a"(val));
39 }
40
41 static int sys_mycall(void) {
42     printk("My syscall is successful!\n");
43     return 12345;
44 }
45
46 static long sys_mycall_with_param(int param) {
47     printk(KERN_INFO "My syscall with param called with %d\n", param);
48     return param + 1; /* 以简单的示例, 返回参数加1 */
49 }
50
51 static int __init init_addsyscall(void) {
52     printk("My syscall is starting...\n");
53     sys_call_table = (unsigned long*)SYS_CALL_TABLE_ADDRESS; /* 使用类型转换 */
54     printk("sys_call_table: 0x%p\n", sys_call_table);
55
56     /* 保存原始系统调用 */
57     anything_saved = (int (*)(void))(sys_call_table[__NR_syscall]);
58     anything_saved_with_param = (long (*)(int))(sys_call_table[__NR_syscall_with_param]);
59
60     /* 设置cr0可更改 */
61     orig_cr0 = clear_and_return_cr0();
62     /* 更改原始的系统调用服务地址 */
63     sys_call_table[__NR_syscall] = (unsigned long)&sys_mycall;
64     sys_call_table[__NR_syscall_with_param] = (unsigned long)&sys_mycall_with_param;
65
66     /* 恢复原始的只读cr0 */
67     setback_cr0(orig_cr0);
68     return 0;
69 }
70
71 static void __exit exit_addsyscall(void) {
72     orig_cr0 = clear_and_return_cr0(); /* 设置cr0中对sys_call_table的更改权限 */
73     sys_call_table[__NR_syscall] = (unsigned long)anything_saved; /* 恢复原有的系统调用 */
74     sys_call_table[__NR_syscall_with_param] = (unsigned long)anything_saved_with_param; /* 恢复原有的带参系统调用 */
75     setback_cr0(orig_cr0); /* 恢复原有的cr0值 */
76     printk("My syscall exit...\n");
77 }
78
79 module_init(init_addsyscall);
80 module_exit(exit_addsyscall);
81
82 MODULE_LICENSE("GPL");
83

```

该代码中实现了两个系统调用,分别为无参调用和有参调用

```
1 static int sys_mycall(void);
2 static long sys_mycall_with_param(int param);
3
```

我们知道控制寄存器 cr0 的第 16 位是写保护位。cr0 的第 16 位置为了禁止超级权限,若清零了则允许超级权限往内核中写入数据,这样我们可以再写入之前,将那一位清零,使我们可以写入。然后写完后,又将那一位复原就行了。因此编写 clean_and_return_cr0 函数来实现该操作

```
1 unsigned int clear_and_return_cr0(void) {
2     unsigned int cr0 = 0;
3     unsigned int ret;
4     asm volatile ("movq %%cr0, %%rax" : "=a"(cr0));
5     ret = cr0;
6     cr0 &= 0xfffeffff; // 将cr0变量值中的第17位清0
7     asm volatile ("movq %%rax, %%cr0" :: "a"(cr0));
8     return ret;
9 }
10
```

其中

```
1 #define __NR_syscall 500
2 #define __NR_syscall_with_param 501
3 #define SYS_CALL_TABLE_ADDRESS 0xfffffffffa0800320ULL
```

是根据系统自身情况来设置的宏常量,需要根据系统情况进行修改

Makefile 内容:

```

1  obj-m:=syscall.o
2  PWD:= $(shell pwd)
3  KERNELDIR:= /lib/modules/$(shell uname -r)/build
4  EXTRA_CFLAGS= -O0
5
6  all:
7      make -C $(KERNELDIR) M=$(PWD) modules
8  clean:
9      make -C $(KERNELDIR) M=$(PWD) clean

```

然后进行编译,终端输入 `sudo make`

```

ubuntu@VM-4-12-ubuntu ~/os_experiment/week5: sudo make
make -C /lib/modules/5.15.0-97-generic/build M=/home/ubuntu/os_experiment/week5 module s
make[1]: Entering directory '/usr/src/linux-headers-5.15.0-97-generic'
CC [M] /home/ubuntu/os_experiment/week5/syscall.o
MODPOST /home/ubuntu/os_experiment/week5/Module.symvers
CC [M] /home/ubuntu/os_experiment/week5/syscall.mod.o
LD [M] /home/ubuntu/os_experiment/week5/syscall.ko
BTF [M] /home/ubuntu/os_experiment/week5/syscall.ko
Skipping BTF generation for /home/ubuntu/os_experiment/week5/syscall.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-5.15.0-97-generic'
ubuntu@VM-4-12-ubuntu ~/os_experiment/week5

```

编译成功后的项目结构:

```

├── a.out
├── Makefile
└── modules.order

```

- └─ Module.symvers
- └─ syscall.c
- └─ syscall.ko
- └─ syscall.mod
- └─ syscall.mod.c
- └─ syscall.mod.o
- └─ syscall.o
- └─ test.c

0 directories, 11 files

然后将系统调用添加到内核模块,终端输入 `sudo insmod syscall.ko` 来实现

检查系统调用是否被添加,输入 `lsmod` 来查看

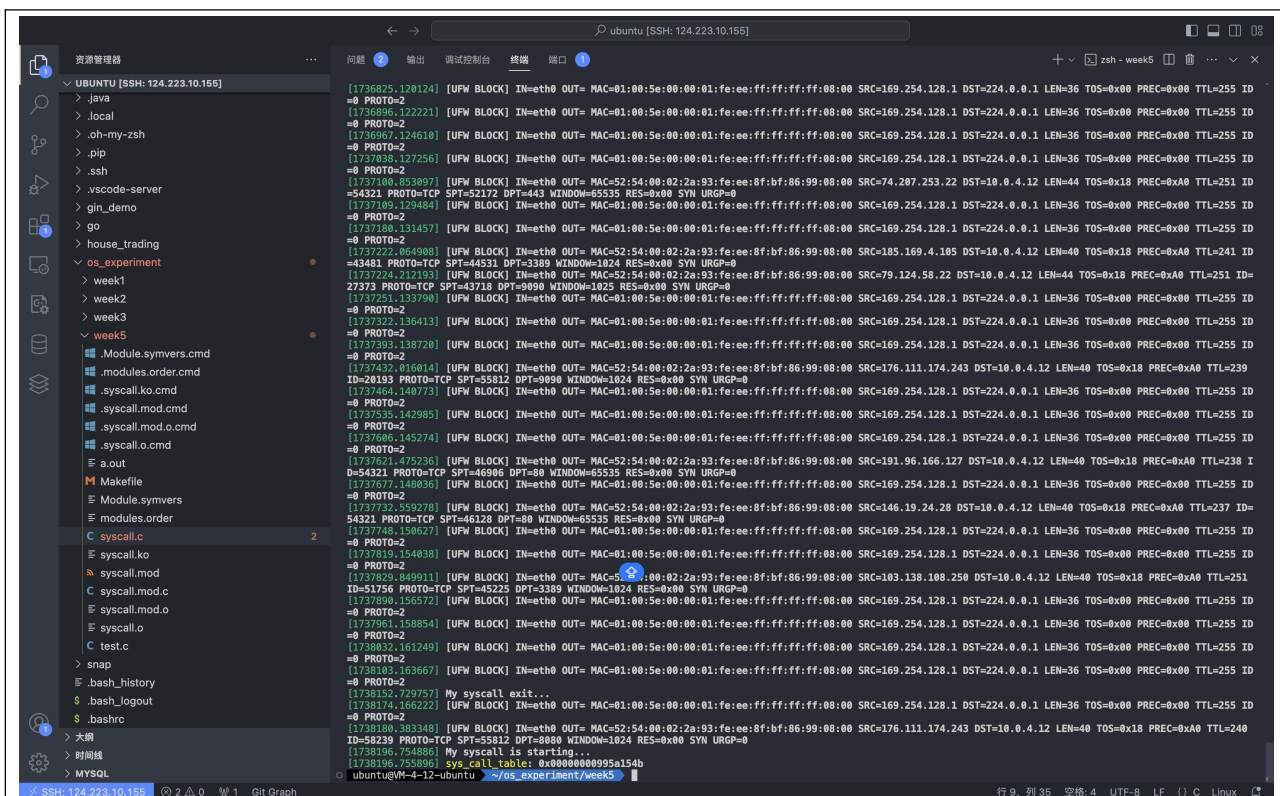
```
ubuntu@VM-4-12-ubuntu: ~/os_experiment/week5
lsmod
Module      Size  Used by
syscall     16384  0
tls         114688  0
xt_recent   24576  0
tcp_diag    16384  0
inet_diag   24576  1 tcp_diag
cpuid       16384  0
iscsi_fc    53248  0
```

可以看到 `syscall` 这个模块,说明该模块被成功添加到了内核

编写测试程序

```
1  #include <unistd.h>
2  #include <sys/syscall.h>
3  #include <stdio.h>
4
5  int main(void)
6  {
7      int test_param = 10;
8      syscall(500);
9      syscall(501, test_param);
10     return 0;
11 }
12
```

查看系统日志,输入 `dmesg`



可以看到

```
[1738196,754886] My syscall is starting...
[1738196,755896] sys_call_table: 0x0000000095a154b
[1738234,684193] [UFW BLOCK] IN=eth0 OUT= MAC=52:54:00:02:2a:93:fe:ee:8f:bf:86:99:08:00 SRC=185.191.126.213 DST=10.0.4.12 LEN=40 TOS=0x18 PREC=0xA0 TTL=251 ID=59664 PROTO=TCP SPT=43840 DPT=80 WINDOW=65535 RES=0x00 SYN URG=0
[1738245,169814] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738316,171659] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738349,221362] [UFW BLOCK] IN=eth0 OUT= MAC=52:54:00:02:2a:93:fe:ee:8f:bf:86:99:08:00 SRC=154.212.141.253 DST=10.0.4.12 LEN=44 TOS=0x18 PREC=0xA0 TTL=238 ID=59664 PROTO=TCP SPT=43840 DPT=80 WINDOW=65535 RES=0x00 SYN URG=0
[1738387,174511] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738458,176777] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738529,179241] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738600,181657] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738636,337925] My syscall exit...
[1738671,184036] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738742,186170] [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:fe:ee:ff:ff:ff:ff:ff:ff:ff:ff:ff SRC=169.254.128.1 DST=224.0.0.1 LEN=36 TOS=0x00 PREC=0x00 TTL=255 ID=0 PROTO=2
[1738769,349095] [UFW BLOCK] IN=eth0 OUT= MAC=52:54:00:02:2a:93:fe:ee:8f:bf:86:99:08:00 SRC=132.148.76.25 DST=10.0.4.12 LEN=40 TOS=0x18 PREC=0xA0 TTL=231 ID=59664 PROTO=TCP SPT=56344 DPT=3389 WINDOW=1024 RES=0x00 SYN URG=0
```

说明该系统调用被成功调用运行

四. 实验总结

本实验旨在通过 Linux 内核修改，实现系统调用的新增，从而加深对系统调用基本原理的理解，并掌握 Linux 系统下系统调用的实现和调用过程。通过实践，我学习了如何在 Linux 环境下通过内核模块法添加新的系统调用，不仅增强了我对操作系统内部工作机制的理解，也提高了我在 Linux 平台下进行系统级编程的能力。